

OSSP (Operating Systems and Systems Programming) –25CS2104E

OSSP LAB – PRACTICAL 5

Aim

To implement inter-process communication and process execution in Linux using system calls such as pipe(), fork(), dup2(), read(), write(), and exec(), by developing a producer-consumer communication system and implementing the equivalent of the shell command `ls -l | grep ".c"`.

Algorithm

1. Start the program.
2. Create an anonymous pipe using pipe().
3. Create a child process using fork().
4. In the parent process, close the reading end of the pipe.
5. Generate data and write it into the pipe using write().
6. In the child process, close the writing end of the pipe.
7. Read the data from the pipe using read().
8. Display the received data as the consumer.
9. Measure the communication time and calculate the communication rate.
10. Close the pipe and terminate both processes.

Task 1

```
bhavana@Bhavana:~$ nano producer_consumer.c
```

```

GNU nano 0.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <time.h>

#define BUFFER_SIZE 100
#define ITEMS 10

int main() {
    int fd[2];
    pid_t pid;

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    /* Parent - Producer */
    if (pid > 0) {

        close(fd[0]);

        int data;

        struct timespec start, end;

        clock_gettime(CLOCK_MONOTONIC, &start);

```

```

GNU nano 0.7.1
        for (int i = 1; i <= ITEMS; i++) {

            data = i * 10;

            write(fd[1], &data, sizeof(data));

            printf("Producer: Generated %d\n", data);
        }

        close(fd[1]);

        wait(NULL);

        clock_gettime(CLOCK_MONOTONIC, &end);

        double time_taken =
            (end.tv_sec - start.tv_sec) +
            (end.tv_nsec - start.tv_nsec) / 1e9;

        printf("\nCommunication Time: %.6f seconds\n",
            time_taken);

        printf("Total items transferred: %d\n", ITEMS);

        if (time_taken > 0) {
            printf("Communication Rate: %.2f items/sec\n",
                ITEMS / time_taken);
        }
    }

    /* Child - Consumer */
    else {

        close(fd[1]);

        int data;

```

```

    int data;

    while (read(fd[0], &data, sizeof(data)) > 0) {

        printf("Consumer: Received %d\n", data);
    }

    close(fd[0]);

    exit(0);
}

return 0;
}

```

```

bhavana@Bhavana:~$ gcc producer_consumer.c -o producer_consumer
bhavana@Bhavana:~$ ./producer_consumer
Producer: Generated 10
Producer: Generated 20
Producer: Generated 30
Producer: Generated 40
Producer: Generated 50
Producer: Generated 60
Producer: Generated 70
Producer: Generated 80
Producer: Generated 90
Producer: Generated 100
Consumer: Received 10
Consumer: Received 20
Consumer: Received 30
Consumer: Received 40
Consumer: Received 50
Consumer: Received 60
Consumer: Received 70
Consumer: Received 80
Consumer: Received 90
Consumer: Received 100

Communication Time: 0.000298 seconds
Total items transferred: 10
Communication Rate: 33503.19 items/sec
bhavana@Bhavana:~$

```

Task 2

Communication Rate: 33803.13 Items/s
bhavana@Bhavana:~\$ nano ls_grep.c

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    int fd[2];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid1 = fork();

    if (pid1 < 0) {
        perror("fork");
        return 1;
    }

    /* First child - ls -l */
    if (pid1 == 0) {

        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    pid_t pid2 = fork();
```

```

if (pid2 < 0) {
    perror("fork");
    return 1;
}

/* Second child - grep ".c" */
if (pid2 == 0) {
    close(fd[1]);

    dup2(fd[0], STDIN_FILENO);

    close(fd[0]);

    execlp("grep", "grep", ".c", NULL);

    perror("execlp grep");
    exit(1);
}

/* Parent */
close(fd[0]);
close(fd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

return 0;
}

```

```

bhavana@Bhavana:~$ gcc ls_grep.c -o ls_grep
bhavana@Bhavana:~$ touch test1.c
bhavana@Bhavana:~$ touch test2.c
bhavana@Bhavana:~$ ls
a.out          history.c      ls_grep        parser.c       process.c.save  project        skill          task1          tokenizer.c
alarmclock     interactive_shell  ls_grep.c     permission_test.txt  process_states  q1.c          skill1.1      task1.c       zombie
command_exec   interactive_shell.c memory.c       practical2.c     process_states.c  readfile      skill3        test1.c       zombie.c
command_exec.c keyboard       memory.c      practical4.c     producer_consumer  readfile.c    skill3.c     test2.c       zombie_fixed
history        keyboard.c    parser        practical4.c     producer_consumer.c  sample.txt    student.txt  tokenizer     zombie_fixed.c

```

```

bhavana@Bhavana:~$ ./ls_grep
drwxr-xr-x 2 bhavana bhavana 4096 Sep 24 04:54 alarmclock
-rwxr-xr-x 1 bhavana bhavana 16272 Jul 30 05:36 command_exec
-rw-r--r-- 1 bhavana bhavana 689 Jul 30 05:31 command_exec.c
-rwxr-xr-x 1 bhavana bhavana 16392 Oct 1 03:18 history
-rw-r--r-- 1 bhavana bhavana 1382 Oct 1 03:17 history.c
-rwxr-xr-x 1 bhavana bhavana 16312 Aug 24 08:14 interactive_shell
-rw-r--r-- 1 bhavana bhavana 575 Aug 24 08:14 interactive_shell.c
-rw-r--r-- 1 bhavana bhavana 989 Aug 24 08:16 keyboard.c
-rwxr-xr-x 1 bhavana bhavana 16304 Oct 1 03:30 ls_grep
-rw-r--r-- 1 bhavana bhavana 972 Oct 1 03:30 ls_grep.c
-rwxr-xr-x 1 bhavana bhavana 16432 Oct 1 03:21 memory
-rw-r--r-- 1 bhavana bhavana 2995 Oct 1 03:21 memory.c
-rwxr-xr-x 1 bhavana bhavana 16312 Oct 1 03:10 parser
-rw-r--r-- 1 bhavana bhavana 1845 Oct 1 03:10 parser.c
-rw-r--r-- 1 bhavana bhavana 1191 Aug 23 13:38 practical2.c
-rwxr-xr-x 1 bhavana bhavana 16344 Sep 19 09:20 practical4
-rw-r--r-- 1 bhavana bhavana 1447 Sep 19 09:20 practical4.c
-rw----- 1 bhavana bhavana 7 Jul 28 06:06 process.c.save
-rwxr-xr-x 1 bhavana bhavana 16560 Aug 24 06:51 process_states
-rw-r--r-- 1 bhavana bhavana 1397 Aug 24 06:57 process_states.c
-rwxr-xr-x 1 bhavana bhavana 16400 Oct 1 03:28 producer_consumer
-rw-r--r-- 1 bhavana bhavana 1491 Oct 1 03:28 producer_consumer.c
drwxr-xr-x 2 bhavana bhavana 4096 Sep 24 04:51 project
-rw-r--r-- 1 bhavana bhavana 59 Aug 18 05:50 q1.c
-rw-r--r-- 1 bhavana bhavana 447 Aug 18 06:08 readfile.c
-rw-r--r-- 1 bhavana bhavana 1391 Sep 21 08:27 skill3.c
-rw-r--r-- 1 bhavana bhavana 690 Aug 1 07:59 task1.c
-rw-r--r-- 1 bhavana bhavana 0 Oct 1 03:31 test1.c
-rw-r--r-- 1 bhavana bhavana 0 Oct 1 03:31 test2.c
-rwxr-xr-x 1 bhavana bhavana 16312 Oct 1 03:07 tokenizer
-rw-r--r-- 1 bhavana bhavana 1602 Oct 1 03:07 tokenizer.c
-rw-r--r-- 1 bhavana bhavana 491 Sep 19 09:22 zombie.c
-rw-r--r-- 1 bhavana bhavana 589 Sep 19 09:31 zombie_fixed.c
bhavana@Bhavana:~$ |

```

Result

The producer-consumer communication system was successfully implemented using an anonymous pipe, where the parent produced data and the child consumed it. The communication time and transfer rate were also measured successfully.

The shell pipeline `ls -l | grep ".c"` was successfully implemented using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls. The output of `ls -l` was successfully transferred through the pipe and filtered by `grep ".c"`.